WIFI-AP Mode Experiment

1. WIFI-AP Mode Overview

WiFi AP (Access Point, Hotspot Mode) allows ESP32-S3 to act as a Wi-Fi hotspot. Other devices (phones/computers) can connect to it. ESP32-S3 internally starts a DHCP server to assign IP addresses, with the default subnet `192.168.4.x` and the AP's own IP at `192.168.4.1`. This experiment monitors connected station count changes in real-time and prints logs.

AP mode has **irreplaceable application value** in the IoT field. First, it is **the foundation of network configuration technology** - when a device is first deployed and no WiFi network is available, the device runs in AP mode first. The user's phone connects to the hotspot, then sends the router password to the device through a webpage or app, and the device switches to STA mode to connect to the network (this is the first step of the SmartConfig / Web Provisioning flow). Second, in **scenarios without a router** (field operations, construction site inspection, temporary exhibitions), AP mode allows users to directly connect to the device point-to-point for parameter configuration, status viewing, and firmware upgrades, without relying on any network infrastructure. Additionally, AP mode is widely used in **local area network multi-person collaboration** scenarios - multiple phones or computers connected to the same ESP32-S3 hotspot to achieve local chat, file sharing, multiplayer games, and other LAN applications.

Typical Application Scenarios:

Application Type	Example
Device Provisioning	Phone connects to ESP32 hotspot → Configure router password via webpage
Direct Control	Phone connects directly to ESP32, control peripherals via HTTP/TCP
Ad-Hoc LAN	Multiple ESP32s form a centerless self-organizing network

2. Core Concepts

2.1 SoftAP Workflow

`WiFi.softAP(ssid, pass)` creates an independent BSS (Basic Service Set), with internal automatic completion of: selecting the most free channel (1-13) → starting WPA2-PSK encryption → starting DHCP server → periodically sending Beacon frames to announce network presence.

2.2 Connection Detection Methods

Two ways to detect device connection/disconnection:

- **Polling** (this experiment): Regularly call `WiFi.softAPgetStationNum()` to compare value changes
- **Event callback**: `WiFi.onEvent()` listens for `ARDUINO_EVENT_WIFI_AP_STACONNECTED` / `STADISCONNECTED` events

---

3. Hardware Dependencies

Pure software experiment. ESP32-S3 has a built-in 2.4GHz Wi-Fi transceiver, and the default maximum number of SoftAP connections is 4.

---

4. Project Architecture and Flow

```
setup()
  ├─ Serial.begin(115200)
  ├─ WiFi.mode(WIFI_AP)               // Pure AP mode
  └─ WiFi.softAP(SSID, PASS)          // Start SoftAP + DHCP

loop()
  ├─ now = WiFi.softAPgetStationNum() // Get current connection count
  ├─ if (now != last)                 // Count changed → Device joined/left
  |     ├─ Print "New device connected" or "Device disconnected"
  |     └─ Print current connection count
  └─ delay(500)
```

---

5. Source Code Details

Source code: `Source code → Arduino → 3.Network Communication → WIFI_AP → WIFI_AP.ino`

5.1 Configuration and Global Variables

```
#include <WiFi.h>
const char* AP_SSID = "ESP32-S3-AP";
const char* AP_PASS = "12345678";       // WPA2 requires at least 8 characters
int current_stations = 0;
```

5.2 `setup()` — Start AP

```
void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_AP);
  WiFi.softAP(AP_SSID, AP_PASS);

  Serial.printf("Hotspot Name: %s\n", AP_SSID);
  Serial.printf("ESP32 AP IP: %s\n", WiFi.softAPIP().toString().c_str());
}
```

> `WiFi.softAPIP()` returns `192.168.4.1` (default), which can be customized via `WiFi.softAPConfig()`. DHCP automatically assigns `192.168.4.x` addresses to connected devices.

5.3 `loop()` — Connection Status Monitoring

```
void loop() {
  int now_stations = WiFi.softAPgetStationNum();

  if (now_stations != current_stations) {
    if (now_stations > current_stations)
      Serial.println("New device has connected to the AP!");
    else
      Serial.println("Device disconnected from the AP!");

    Serial.printf("Number of currently connected devices: %d\n", now_stations);
    current_stations = now_stations;
  }
  delay(500);
}
```

> Polling is simple but has a maximum delay of 500ms. For scenarios requiring high real-time performance, it is recommended to use the `WiFi.onEvent()` event-driven approach.

---

6. Operation Flow

6.1 Build and Flash

Arduino flashing process: See `Arduino → 1. Before Development → 2. Build and Flash`.

1. Build and flash
2. Search for "ESP32-S3-AP" on phone/computer Wi-Fi, connect with password `12345678`

6.2 Normal Phenomena

Serial output:

```
==================================
 WiFi AP Started successfully
 Hotspot Name: ESP32-S3-AP
 Hotspot Password: 12345678
 ESP32 AP IP: 192.168.4.1
==================================
New device has connected to the AP!
Number of currently connected devices: 1
```

Connection success:

Disconnection:

6.3 Verification Methods

Operation	Expected Result
Phone connects to hotspot	Serial prints "New device connected", count +1
Phone disconnects from hotspot	Serial prints "Device disconnected", count -1
Multiple devices connected simultaneously	Count increases cumulatively

---

7. Common Issues

Phenomenon	Cause	Solution
Phone cannot find hotspot	2.4GHz channel conflict with router	ESP32 only supports 2.4GHz, confirm phone support
Cannot access internet after connection	AP mode does not include WAN exit	Normal phenomenon - AP does not provide internet access
Cannot connect more than 4 devices	Default maximum 4 STAs	`WiFi.softAPConfig()` can modify, maximum 10